All-Pairs-Shortest-Path

Materia	Algoritmi
Anno	Secondo Anno
Data	@January 10, 2025
Studiato	Done

Ricordiamo che con il simbolo $l_{ij}^{(r)}$ indichiamo il peso di un cammino minimo da i a j che contiene al più r archi. Possiamo dire che:

$$l_{ij}^{(0)} = \begin{cases} 0 & i = j \\ \infty & i \neq j \end{cases}$$

Se $r \geq 1$, consideriamo un cammino che va da i a k con $r-1$ archi e poi considerare l'arco (k, j), ottenendo:

$$l_{ij}^{(r)} = \min_{1 \leq k \leq n} \{ l_{ik}^{(r-1)} + w_{kj} \}$$

E poiché un cammino minimo è semplice e non ci sono cicli negativi allora si ha:

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \dots$$

in quanto si hanno al più $n-1$ archi. Definiamo la matrice:

$$L^{(r)} = (l_{ij}^{(r)})$$

Ora possiamo definire un algoritmo che si basa sulla programmazione dinamica.

Algoritmo per All-Pairs-Shortest-Path basato sulla programmazione dinamica e la moltiplicazione tra matrici

Presa in input la matrice $W = (w_{ij})$, calcoliamo una serie di matrici $L^{(0)}, L^{(1)}, \dots, L^{(n-1)}$ con $L^{(r)} = (l_{ij}^{(r)})$ per $r = 0, \dots, n-1$. La matrice finale

$L^{(n-1)}$ conterrà i pesi dei cammini minimi e sarà una soluzione al problema.

Sappiamo che:

$$l_{ij}^{(0)} = \begin{cases} 0 & i = j \\ \infty & i \neq j \end{cases}$$

Per calcolare $l_{ij}^{(r)} = \min_{1 \leq k \leq n} \{ l_{ik}^{(r-1)} + w_{ij} \}$, esiste una relazione tra il calcolo di $L^{(r)}$ a partire da $L^{(r-1)}$ e W e la moltiplicazione tra matrici. Il prodotto $A \cdot B$ può essere visto come:

$$\sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

Se sostituiamo alla somma il prodotto e all'operazione di minimo la somma, si ottiene che $l_{ij}^{(r)}$ è uguale all'elemento di posto i, j del prodotto $L^{(r-1)} \cdot W$.
Significa quindi che **la correttezza di tale procedura si basa sul semianello tropicale, in cui l'operazione di somma** (intesa come nella teoria dei gruppi) **è l'operazione di minimo e l'operazione di prodotto è la somma algebrica**.
Il cuore dell'algoritmo è la procedura $\text{Extend-Shortest-Path}$ che estende di un arco i cammini trovati all'iterazione precedente.

```
Extend-Shortert-Paths(L^(r-1), W, L^(r), n):
   for i = 1 to n do
      for j = 1 to n do
         for k = 1 to n do
            l_ij^(r) = min(l_ik^(r-1)+w_kj, l_ij^(r))
```

La procedura ha complessità $\Theta(n^3)$.

Possiamo risolvere All-Pairs-Shortest-Path moltiplicando ripetutamente delle matrici. Infatti:

$$L^{(1)} = L^{(0)} \cdot W = W$$
$$\dots$$
$$L^{(n-1)} = L^{(n-2)} \cdot W = W^{(n-1)}$$

L'algoritmo Slow-APSP calcola queste sequenze di matrici in tempo $\Theta(n^4)$:

```
Slow-APSP(W, L^(0), n):
   L = (l_ij), M = (m_ij)
   L = L(0)
   for r = 1 to n-1 do
     M = (infin)
     Extend-Shortest-Paths(L, W, M, n) // calcola "M = L x W"
     L = M
   return L
```

Possiamo migliorare la procedura utilizzando una tecnica basata sui quadrati successivi: infatti non ci serve sapere ogni termine $L^{(i)}$, ma possiamo notare che $L^{(2)} = W^2$, $L^{(4)} = W^2 \cdot W^2 = W^4$ e così via. Si ottiene così un numero di prodotti pari a $\lceil \log(n-1) \rceil$. La procedura diventa:

```
Fast-APSP(W, n):
   L = new matrix(n, n)
   M = new matrix(n, n)
   L = W
   r = 1
   while r < n - 1 do
      M = (infin)
      Extend-Shortest-Paths(L, L, M, n)
      r = 2r
      L = M
   return L
```

Impiegando $\Theta(n^3 \log n)$.

Algoritmo di Floyd-Warshall

L'**algoritmo di Floyd-Warshall** risolve il problema APSP impiegando un tempo $\Theta(n^3)$. Esso si basa su una definizione ricorsiva diversa del problema.

> *Un vertice intermedio di un cammino semplice $p =< v_1, v_2, ..., v_l >$ è qualunque vertice appartenente all'insieme $\{ v_2, v_3, ..., v_{l-1} \}$.*

Enumeriamo i vertici di G, ossia $V = \{ 1, 2, ..., n \}$ e consideriamo il sottoinsieme $\{ 1, 2, ..., k \}$ di vertici per qualche $1 \leq k \leq n$. Non è un sottoinsieme qualunque: infatti non abbiamo mai il sottoinsieme $\{ 2, 4, ... \}$ o $\{ 1, 3, 9... \}$ e sequenze diverse da numeri consecutivi.

Per ogni coppia di vertici (i, j) consideriamo tutti i cammini minimi da i a j i cui vertici intermedi apaprtengono all'insieme $\{ 1, 2, ..., k \}$. Tra questi cammini minimi, consideriamo p. Chiaramente p è un cammino semplice assumendo non vi siano cicli negativi (che è ipotesi del problema). Consideriamo il nodo k:

- se k non è un vertice intermedio allora possiamo dire che ogni vertice intermedio di p appartiene a $\{ 1, ..., k-1 \}$;
- se k è un vertice intermedio di p, allora decomponiamo p in due cammini semplici p_1, p_2, con $p_1 = i \rightsquigarrow k$ e $p_2 = k \rightsquigarrow j$, e k non è un vertice intermedio di nessuno dei due cammini semplici. I vertici intermedi di p_1, p_2 appartengono a $\{ 1, ..., k-1 \}$.

Questa osservazione suggerisce una formulazione ricorsiva diversa da quella introdotta precedentemente. Sia $d_{ij}^{(k)}$ il peso di un cammino minimo da i a j i cui vertici intermedi appartengono a $\{ 1, ..., k \}$. Allora si ha che:

$$d_{ij}^{(0)} = w_{ij}$$

in quanto con $k = 0$ non ci sono vertici intermedi, quindi da i a j passiamo tramite un arco se esso esiste. Allora:

$$d_{ij}^{(k)} = \min\{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$$

La matrice $D^{(n)} = (d_{ij}^{(n)})$ darà la soluzione al problema visto che i vertici intermedi di un cammino minimo qualunque appartengono all'insieme $\{ 1, 2, ..., n \}$. Diremo allora che $d_{ij}^{(n)} = \delta(i, j)$ per ogni vertice $i, j \in V$.
L'algoritmo Floyd-Warshall calcola i valori $d_{ij}^{(k)}$ al crescere di k in maniera bottom up:

```
Floyd-Warshal(W, n):
    D^(0) = W
    for k = 1 to n do
        D^(k) = (d_ij^(k))          // nuova matrice di ordine n x n
        for i = 1 to n do
            for j = 1 to n do
                d_ij^k = min(d_ij^(k-1), d_ik^(k-1)+d_kj^(k-1))
    return D^(n)
```

E il tempo computazionale è $\Theta(n^3)$.
La funzione ricorsiva ogni volta controlla due cose:

- il cammino minimo che va da i a j che usa vertici intermedi in $\{ 1, 2, ..., k - 1 \}$;
- il cammino minimo che va da i a j dividendo il cammino minimo in due cammini minimi.

Ricostruzione del cammino minimo

Ci sono diversi modi per ricostruire il cammino minimo. Possiamo calcolare a posteriori la matrice dei predecessori Π. Data questa matrice usiamo la funzione $\text{Print-All-Pairs-Shortest-Paths}$.
Alternativamente, la matrice
Π può essere calcolata nel corso dell'algoritmo, calcolando $\Pi^{(i)}$ contestualmente a $D^{(i)}$ per $i = 0, ..., n$. Avremo che $\Pi = \Pi^{(n)}$.

Avremo che $\pi_{ij}^{(k)}$ è definito come il predecessore di j in un cammino minimo da i a j i cui vertici intermedi sono in $\{ 1, ..., k \}$. Formalmente, calcoliamo:

$$\pi_{ij}^{(0)} = \begin{cases} \text{nil} & i = j \vee w_{ij} = \infty \\ i & i \neq j \wedge w_{ij} < \infty \end{cases}$$

Se $k \geq 1$ allora si verifica:

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \\ \pi_{kj}^{(k-1)} & d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$